Feuille de TD n°3

Dans cette feuille et les suivantes, il faut systématiquement prouver la terminaison et correction de tous les algorithmes. Il est également demandé de reprendre les feuilles précédentes et de rédiger soigneusement toutes les terminaisons et corrections.

Exercice 1. *1. Écrire une fonction itérative prenant en entrée deux entiers m et n naturels et renvoyant l'entier $m \times n$, sans utiliser le produit ni même la somme. La seule chose que l'on pourra utiliser est une fonction « successeur » $s : \mathbb{N} \to \mathbb{N}$ qui prend en entrée un entier naturel et renvoie le nombre naturel suivant.*

2. Prouver la terminaison et la correction de la fonction

Exercice 2. *La fonction suivante prend une liste d'entiers. Termine-t-elle ? Que fait-elle ?*

```
def f(L):
  if L == []:
    return 0
  return L[0] + f(L[1:])
```

Exercice 3. *La fonction suivante prend une liste d'entiers. Termine-t-elle ? Que fait-elle ?*

```
def b(L):
  n = len(L)
  if n <= 1:
    return L
  for i in range(n-1):
    if L[i] > L[i+1]:
      temp = L[i]
      L[i] = L[i+1]
      L[i+1] = temp
  return L
```

Exercice 4. *Écrire une fonction récursive* `contient(L, x)` *qui renvoie* `True` *si x apparaît dans la liste L, et* `False` *sinon.*

Exercice 5. *Écrire une fonction récursive* `inverse(L)` *qui prend une liste L et renvoie une nouvelle liste contenant les éléments de L dans l'ordre inverse.*

Exercice 6. *La fonction suivante prend en entrée une liste non vide de nombres. Que fait-elle ?*

```
def mystere(L):
    if len(L) <= 1:
        return True
    return (L[0] <= L[1]) and mystere(L[1:])
```

Exercice 7. *La fonction suivante prend en entrée une liste non vide de nombres. Que fait-elle ?*

```
def g(L):
    if len(L) == 1:
        return L[0]
    m = g(L[1:])
    if L[0] > m:
        return L[0]
    return m
```

Exercice 8. *La fonction suivante prend en entrée une liste d'entiers* ***positifs****. Que fait-elle ?*

```
def f(L):
    if L == []: return 1
    if L[0] == 0: return f(L[1:])
    return L[0] * f([L[0] - 1] + L[1:])
```

Exercice 9. *La fonction suivante prend en entrée une liste non vide de nombres. Elle renvoie deux valeurs.*

1. *Prouver la terminaison.*
2. *Dire ce qu'elle renvoie sur une liste* $[a_0, a_1, \cdots a_n]$*, sans justifier.*
3. *Justifier la réponse précédente.*

```
def f(L):
    if len(L) == 1:
        return (0, 1)
    s, p = f(L[:-1])
    return (L[-1]*p + s, p + 1)
```

Exercice 10. *On considère une liste* L *d'entiers. Une* inversion *dans cette liste est un couple d'entiers* (i, j) *tels que* $i < j$ *et* $L[i] > L[j]$*.*

Écrire une fonction Python `nb_inversions(L)` *prenant une liste (de nombres) en entrée et retournant le nombre d'inversions dans la liste.*